🚀 Day 18: PySpark Aggregations, Groupings & DataFrame Joins

21-Day Data Engineering Sprint • The Data Refinery Masterclass

Author: Prasad S Hiremath | Technical Curriculum & Hands-on Lab

💡 Daily Motivation & Philosophy:

"Focus starts with elimination, improves with concentration, and compounds with continuation."

"I am not afraid of a person who knows 10,000 different kicks, I am afraid of a person who practiced a single kick 10,000 times." — **Bruce Lee**

"In the same span of time, one individual builds results through deliberate action, while another remains idle, trapped in overthinking. Success is not defined by the time available, but by how purposefully it is used."

Welcome to Day 18! You've mastered ingestion and column transformations. Today, we conquer relational data grouping, complex aggregations, and distributed table joins—the bedrock of analytics and reporting. Let's compound our skills!

🏢 The Real-World Scenario: The 50-Million-Row Join Bottleneck at "InstaDrop"

At **InstaDrop**, management requested a daily report linking 50 million transaction records (`orders`) with 200,000 user profiles (`customers`) to analyze revenue by city.

A junior engineer wrote this standard join:

```
# ⚠️ THE NAIVE SHUFFLE JOIN:
result_df = orders_df.join(customers_df, "customer_id", "inner")
```

What happened in production?

1. Spark triggered a **Shuffle**, moving tens of gigabytes of data across network partitions between worker nodes to match `customer_id` keys.
2. The job took 45 minutes and nearly crashed the cluster due to network congestion and disk spillover!

Key Takeaway: A senior data engineer recognized that `customers_df` fits in memory. By using a **Broadcast Join**, the small table is copied to every worker node in advance, cutting runtime from 45 minutes down to **45 seconds**!

📖 Part 1: Core Concepts Explained Simply

1️⃣ Grouping and Aggregations (`groupBy` & `.agg()`)

When you want to summarize data (e.g., total sales per store, average delivery time per city), you use `groupBy()` followed by aggregation expressions in `.agg()`.

- `groupBy()`: Divides data into logical buckets based on column values.
- `.agg()`: Applies aggregation functions (`sum`, `avg`, `max`, `min`, `count`, `countDistinct`).

```
from pyspark.sql.functions import sum, avg, max, col

# Calculate total and average order amount per customer status
summary_df = df.groupBy("status").agg(
    sum("amount").alias("total_revenue"),
    avg("amount").alias("average_order_value")
)
```

2️⃣ PySpark Joins (`join()`)

Just like SQL, PySpark allows you to combine two DataFrames based on common keys:

- **`inner`:** Keeps only rows where the join key exists in *both* DataFrames.
- **`left` (or `left_outer`):** Keeps all rows from the left DataFrame, plus matching rows from the right (unmatched right columns become `null`).
- **`right` (or `right_outer`):** Keeps all rows from the right DataFrame, plus matching rows from the left.
- **`full` (or `outer`):** Keeps all rows from both DataFrames.

3️⃣ The Secret Weapon: Broadcast Joins (`broadcast()`)

When joining a **large table** with a **small reference table**, standard joins force an expensive shuffle across the network. Wrapping the small table in `broadcast()` sends a copy to every worker node, enabling local lookups and bypassing the shuffle entirely.

```
from pyspark.sql.functions import broadcast

# Supercharge performance for small reference tables
optimized_df = orders_df.join(broadcast(customers_df), on="customer_id", how="inner")
```

📋 WhatsApp Quick Reference Table

Operation	Python PySpark Syntax	Production Purpose
Group & Aggregate	`df.groupBy("col").agg(sum("val"))`	Computes summary metrics across partitioned groups.
Alias Aggregated Column	`.alias("total_sales")`	Renames calculated metrics for clean reporting.
Standard Inner Join	`df1.join(df2, "id", "inner")`	Retains strictly matching relational keys.
Left Outer Join	`df1.join(df2, "id", "left")`	Preserves all left table rows with optional right matches.
Performance Broadcast	`df1.join(broadcast(df2), "id")`	Eliminates network shuffling for small dimension tables.

💻 Part 2: Beginner-Level Coding Challenge & Full Solution

Scenario: Join InstaDrop `orders` and `customers` datasets, associate each order with the customer's city, aggregate total revenue, and find the highest single order amount per city using a broadcast join.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import sum, max, col, broadcast

# 1. Initialize SparkSession
spark = SparkSession.builder     .appName("InstaDrop_Day18_Aggregations_Joins")     .master("local[*]")     .getOrCreate()

# 2. Mock Datasets
orders_data = [
    ("ORD-1", 101, 500.0),
    ("ORD-2", 102, 1200.0),
    ("ORD-3", 101, 300.0),
    ("ORD-4", 103, 750.0),
    ("ORD-5", 102, 450.0)
]
orders_schema = ["order_id", "customer_id", "amount"]
orders_df = spark.createDataFrame(orders_data, schema=orders_schema)

customers_data = [
    (101, "Bengaluru"),
    (102, "Belagavi"),
    (103, "Mysuru")
]
customers_schema = ["customer_id", "city"]
customers_df = spark.createDataFrame(customers_data, schema=customers_schema)

# 3. Perform Optimized Broadcast Join
joined_df = orders_df.join(broadcast(customers_df), on="customer_id", how="inner")

# 4. Group by City and Aggregate
summary_df = joined_df.groupBy("city").agg(
    sum("amount").alias("total_revenue"),
    max("amount").alias("highest_single_order")
)

# 5. Show Results
print("--- InstaDrop City Revenue Summary ---")
summary_df.show()

spark.stop()
```

Execution Output:

```
--- InstaDrop City Revenue Summary ---
+---------+-------------+--------------------+
|     city|total_revenue|highest_single_order|
+---------+-------------+--------------------+
| Belagavi|       1650.0|              1200.0|
|Bengaluru|        800.0|               500.0|
|   Mysuru|        750.0|               750.0|
+---------+-------------+--------------------+
```

❓ Part 3: 5 Core Concept MCQs with Detailed Solutions

CORE QUESTION 1

When performing grouped analytics in PySpark, what is the role of .agg() when paired with groupBy()?

A) It deletes duplicate rows across the entire cluster.
B) It allows you to apply one or more aggregate expressions (such as sum, avg, max) to each group.
C) It sorts the DataFrame columns in alphabetical order.
D) It converts the DataFrame into a Pandas object.

🔑 **Correct Answer: B**
Detailed Reasoning: `groupBy()` partitions your data into logical buckets, and `.agg()` accepts aggregation functions to compute summary metrics (e.g., total sum, average) across those buckets.

CORE QUESTION 2

In a PySpark join with how="inner", what happens to records in df1 that do not find a matching key in df2?

A) They are included in the result with null values in the right-side columns.
B) They are dropped entirely from the resulting DataFrame.
C) They cause the Spark job to fail with a KeyNotFound exception.
D) They are automatically moved to the broadcast table.

🔑 **Correct Answer: B**
Detailed Reasoning: An inner join strictly retains only the records where the join key exists in *both* datasets. Unmatched rows on either side are filtered out.

CORE QUESTION 3

Why do senior data engineers use broadcast() when joining a massive transaction table with a small dimension table?

A) It compresses both tables into ZIP format to save disk space.
B) It eliminates expensive network shuffling by copying the small table to all worker nodes.
C) It forces Spark to execute the code using a single thread on the driver.
D) It converts string columns into integers automatically.

🔑 **Correct Answer: B**
Detailed Reasoning: Shuffling large datasets across the network is the #1 performance bottleneck in distributed computing. Broadcasting a small table pushes a local copy to every executor, enabling map-side joins without shuffling.

CORE QUESTION 4

If you perform a left join (how="left"), what value does Spark assign to columns from the right-hand DataFrame when there is no matching key?

A) 0
B) "" (empty string)
C) null (None)
D) -1

🔑 **Correct Answer: C**
Detailed Reasoning: In outer joins (`left` , `right` , `full`), when a record on the primary side has no match on the opposing side, Spark populates the missing columns with `null` .

CORE QUESTION 5

Why is it considered best practice to use .alias("metric_name") after an aggregation function like sum("amount")?

A) It prevents Spark from crashing due to memory overflows.
B) It renames the resulting cryptic column name (e.g., sum(amount)) to a clean, readable name for downstream reporting.
C) It encrypts sensitive financial data.
D) It converts decimal numbers into integers.

🔑 **Correct Answer: B**
Detailed Reasoning: Default aggregate column names in Spark look like `sum(amount)` . Using `.alias()` ensures clean, professional column headers for downstream data lake storage and reporting dashboards.

🌟 Part 4: 3 Beginner-Friendly Bonus Q&As

BONUS QUESTION 6

What is a "Shuffle" in Apache Spark, and why should we minimize it?

💡 **Detailed Answer & Reasoning:**
A shuffle occurs when Spark needs to redistribute data across different worker nodes so that all records with the same key end up on the same executor (e.g., during wide transformations like `groupBy` or non-broadcast `join`). This involves writing data to disk, sending it across network wires, and reading it back on the receiving node. It is an expensive I/O and network operation that slows down pipelines.

BONUS QUESTION 7

Can you join on multiple columns in PySpark?

💡 **Detailed Answer & Reasoning:**
Yes! You can pass a list of column names to the `on` parameter: `df1.join(df2, on=["customer_id", "store_id"], how="inner")` . Spark will ensure all specified keys match before combining the rows.

BONUS QUESTION 8

What is the difference between count() and countDistinct() in aggregations?

💡 **Detailed Answer & Reasoning:**

`count("order_id")` counts the total number of rows (including duplicate order occurrences), whereas `countDistinct("customer_id")` counts only the unique, non-repeating customer IDs within that group, ignoring duplicate occurrences.